CodeNicely Shortlisting Process

CodeNicely Software Services LLP

Centralized Authentication & RBAC for Microservices

⏳ Duration

6 Days

> This is a **design-heavy and security-focused assignment**, not a race. Candidates are expected to make thoughtful architectural decisions, justify tradeoffs, and demonstrate real-world backend engineering practices.

🎯 Objective

Design and implement a **Centralized Authentication & Authorization system** that supports **Role-Based Access Control (RBAC)** for a **microservices architecture**.

The system must:

- Centralize **user identity & permissions**
- Issue and validate secure access tokens
- Enforce authorization consistently across services
- Avoid tight coupling between microservices

🧩 Problem Context

In modern microservice architectures:

- Authentication **must not be duplicated** across services
- Authorization rules must be **consistent and scalable**
- Resource services should **not manage users or credentials**

Your task is to build:

- An **Auth Service** responsible for identity & access
- One **Protected Resource Service** that relies on the Auth Service

---

🏗️ High-Level Architecture

```
[ Client ]
    |
    |  Login
    v
[ Auth Service ]
    |
    |  JWT Access Token
    v
[ Resource Service ]
```

Minimum requirement:

- **1 Auth Service**
- **1 Resource Service**

  You may use a monorepo or multiple repositories.

🛠️ Technical Stack

Mandatory

- Node.js (>=18)
- Express.js or Fastify
- REST APIs
- JWT-based authentication

Optional (Preferred)

- TypeScript
- PostgreSQL / MySQL / MongoDB
- Prisma / TypeORM / Sequelize / Mongoose
- Redis
- Docker / Docker Compose
- Swagger / OpenAPI

Candidates **must explain their tech choices** in the README.

👥 Identity & Access Model

1️⃣ User

```
{
  id: string
  email: string
  password: string (hashed)
  isActive: boolean
  createdAt: Date
}
```

2️⃣ Role

```
{
  id: string
  name: string        // admin, manager, user
  description: string
}
```

3️⃣ Permission

```
{
  id: string
  resource: string    // orders, reports, users
  action: string      // read, write, delete
}
```

Relationships

- User ↔ Role → **Many-to-Many**
- Role ↔ Permission → **Many-to-Many**

🔐 Auth Service Requirements

Core Responsibilities

- User authentication
- Role & permission management
- Token issuance
- Token validation strategy

Auth Endpoints

POST /auth/login
POST /auth/refresh (optional but recommended)
POST /auth/logout (optional)

Authentication Rules

- Passwords must be **securely hashed**
- Access tokens must be **short-lived**
- Refresh tokens (if used) must be stored securely
- Disabled users must not receive tokens

🛡️ Authorization (RBAC)

Rules

- Authorization is **permission-based**
- Roles are mapped to permissions
- Resource services must **never** query the Auth DB
- Authorization is enforced via middleware

Permission Format

```
<resource>:<action>
examples:
orders:read
orders:write
orders:delete
```

📦 Resource Service (Protected Service)

Create **one example microservice** (e.g. Orders Service).

Sample Endpoints

GET /orders

POST /orders

DELETE /orders/:id

Required Permissions

Endpoint	Permission
GET /orders	orders:read
POST /orders	orders:write
DELETE /orders/:id	orders:delete

Authorization Rules

- Requests without valid tokens → **401 Unauthorized**
- Requests with insufficient permission → **403 Forbidden**
- Token must be validated before accessing routes

🔄 Token Validation Strategy

Choose **one approach** and justify it:

Option A – Public Key Validation (Recommended)

- Auth Service signs JWT using private key
- Resource Service validates using public key

Option B – Token Introspection

- Resource Service calls Auth Service to validate tokens

⚠️ Error Handling & Security

Expected:

- Centralized error handling
- Proper HTTP status codes
- Clear error messages
- Secure secret management
- Token expiration handling
- No sensitive data leakage

🧠 Architecture Expectations

Demonstrate:

- Clean separation of concerns
- Middleware-based auth logic
- Environment-based config
- Reusable utilities

Suggested Folder Structure

Auth Service

```
auth-service/
  ├── routes/
  ├── controllers/
  ├── services/
  ├── models/
  ├── middleware/
  ├── utils/
  └── app.js
```

Resource Service

```
resource-service/
  ├── routes/
  ├── controllers/
```

```
├── middleware/
└── app.js
```

⭐ Bonus (Strongly Encouraged)

Any of the following earns **extra credit**:

- Token revocation / blacklist
- Redis for refresh tokens
- API Gateway integration
- Policy-based access (RBAC → ABAC)
- Multi-tenant RBAC
- Audit logs (who accessed what)
- Rate limiting
- Docker Compose setup

📄 Submission Requirements

Candidate must provide:

1. GitHub repository (or repositories)
2. README containing:
    - Architecture explanation
    - Auth & RBAC flow
    - Setup instructions
    - Environment variables
    - Security decisions
3. `.env.example`
4. API documentation (Swagger or Postman preferred)

📝 Evaluation Criteria

Category	Evaluation Focus
System Design	Correct microservice separation
Auth Security	Token handling & hashing

Category	Evaluation Focus
RBAC Model	Clean & extensible
Code Quality	Maintainability & clarity
Documentation	Clear technical reasoning

⏳ Timeline: 7 Days

Best of Luck for your Assignment!